Thinking in Parallel 101

Saad NOUFEL
TICLab
UIR

September 8, 2026

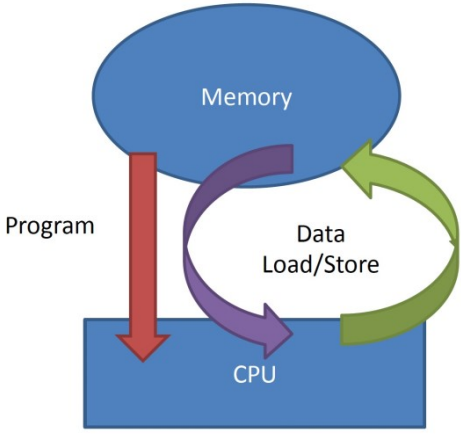
What is Parallel Computing?

Definition

Simultaneous execution of multiple calculations or processes.

Key Concept

Instead of doing tasks one after another (serial), we do multiple tasks at the same time (parallel)



A problem is broken into a discrete series of instructions.
Instructions are executed one after another.
Only one instruction may execute at any moment in time.

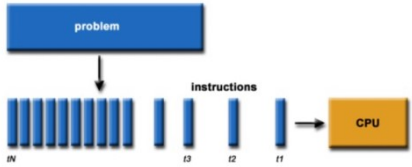


Figure: Serial Computing

Parallel Programming

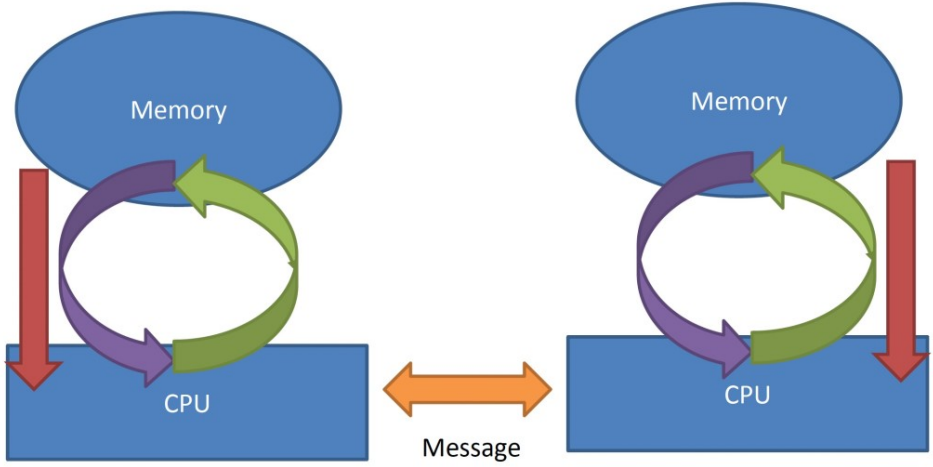


Figure: Parallel Computing

Types of Parallel Computing

Data Parallelism

Distribute data across computing nodes, performing the same operation on each piece of data. Example: Decrypting multiple shanks of data using the same function.

Task Parallelism

Distribute different tasks across multiple computing nodes. Each node preforms different operations. Example: One node reads data, another processes, another writes results.

Type of Parallelism

- **Functional (or task) parallelism**: different people are performing different task at the same time



- **Data Parallelism**: different people are performing the same task, but on different equivalent and independent objects



Figure: Visual example of parallelism.

Flynn's Taxonomy - Classifying Parallel Architectures

Flynn's Taxonomy

A fundamental classification of computer architectures based on instruction and data streams.

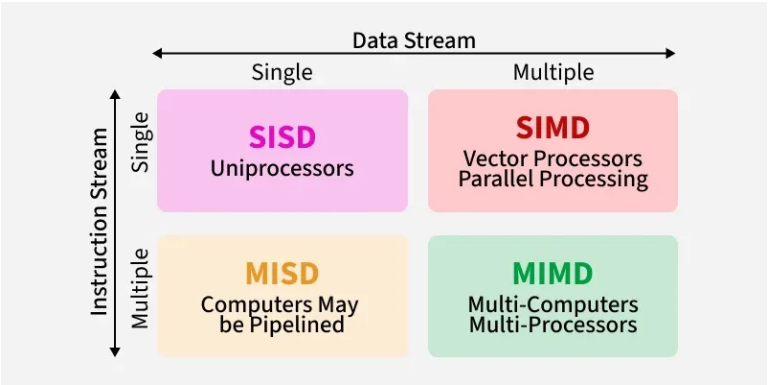
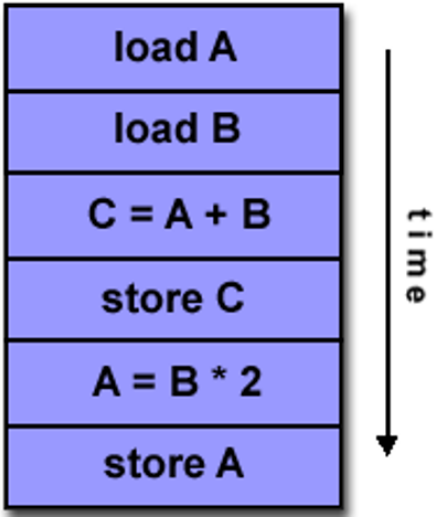
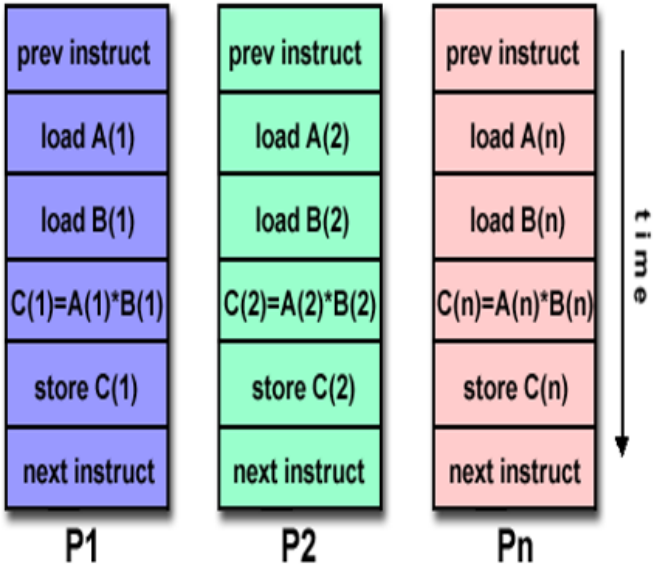


Figure: Classification of Flynn's Taxonomy (geeksforgeeks

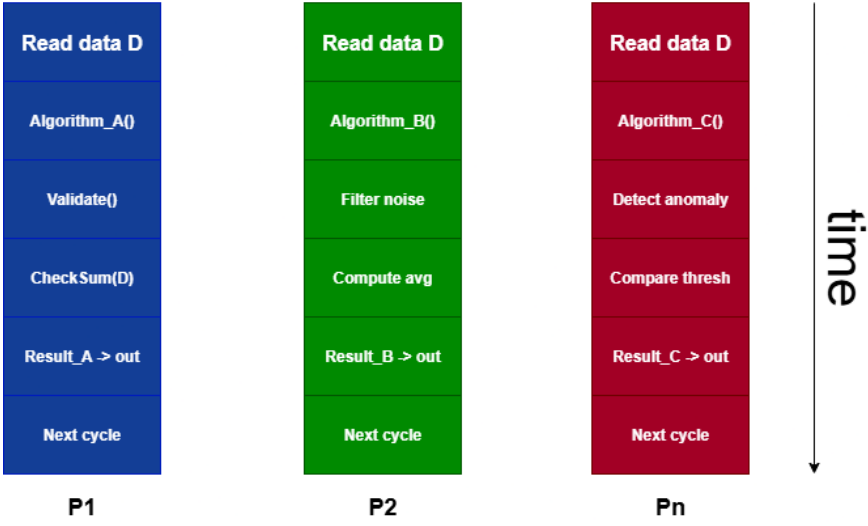
SISD



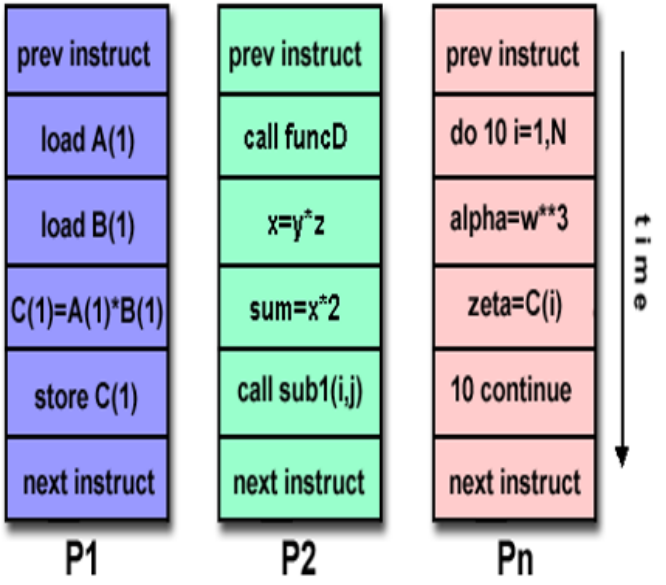
SIMD


prev instruct
load A(1)
load B(1)
C(1)=A(1)*B(1)
store C(1)
next instruct
P1
prev instruct
load A(2)
load B(2)
C(2)=A(2)*B(2)
store C(2)
next instruct
P2
prev instruct
load A(n)
load B(n)
C(n)=A(n)*B(n)
store C(n)
next instruct
Pn
time

MISD


Read data D
Algorithm_A()
Validate()
CheckSum(D)
Result_A -> out
Next cycle
P1
Read data D
Algorithm_B()
Filter noise
Compute avg
Result_B -> out
Next cycle
P2
Read data D
Algorithm_C()
Detect anomaly
Compare thresh
Result_C -> out
Next cycle
Pn
time

MIMD


prev instruct
load A(1)
load B(1)
C(1)=A(1)*B(1)
store C(1)
next instruct
P1
prev instruct
call funcD
x=y*z
sum=x*2
call sub1(i,j)
next instruct
P2
prev instruct
do 10 i=1,N
alpha=w**3
zeta=C(i)
10 continue
next instruct
Pn
time

Parallel Computing Architectures

Multi-core Computing

Multiple processing units (cores) on a single chip. Cores share memory and peripherals. Example: Your laptop with 4, 8, or 16 cores.

Distributed Computing

Multiple autonomous computers connected through a network. Each computer has its own memory and processors Example: Computer clusters, cloud computing.

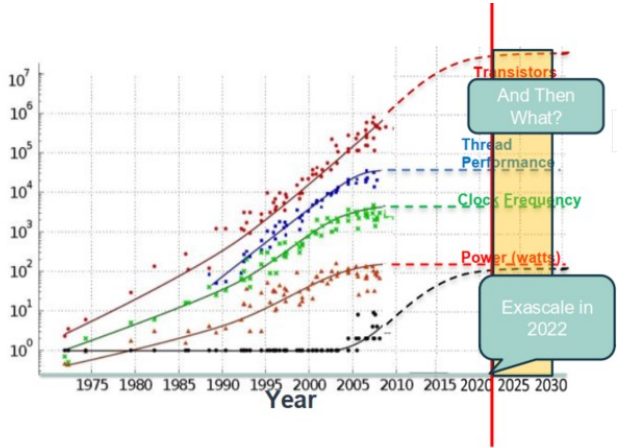
Super-computing

High-performance computing systems designed for complex and large-scale computations. Combines thousands of processors Example: Al Capitain, Toubkal HPC.

The Foundation - Moore's Law

Moore's Law (1965)

The number of transistors in a dense integrated circuit (IC) doubles about every two years.



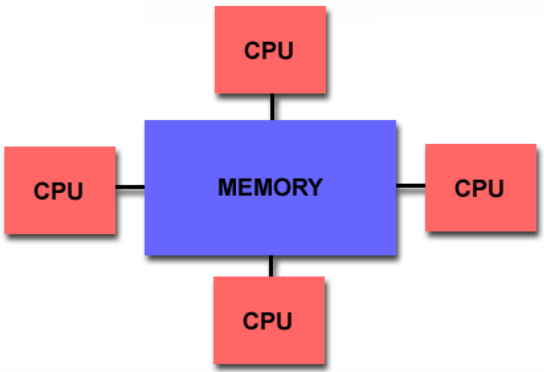
The End of Moore's Law - Why We Need Parallel Thinking

The Three Walls That Ended the Free Lunch:

- The Power Wall ($\sim$ 2005)
  - Clock speeds stopped scaling due to heat dissipation limits
- The Memory Wall
  - Processor speed improved 1000x faster than memory speed Memory latency became the bottleneck
- The ILP Wall (Instruction-Level Parallelism)
  - Most programs have only 2-3 instructions that can execute simultaneously

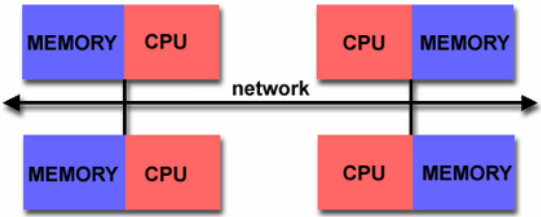
Parallel Computer Memory Architectures: Shared Memory Architecture

- All processors access all memory as a single global address space.
- Data sharing is fast (Low latency).
- Lack of scalability between memory and CPUs.



Parallel Computer Memory Architectures: Distributed Memory

- Each processor has its own memory.
- Is scalable, no overhead for cache coherency.
- Programmer is responsible for many details of communication between processors.



Parallel Performance

Serial Time

Time for a serial program to solve a problem with an input of size n, denoted $T(S)$.

Parallel Time

Time for a parallel program to solve the same problem with an input size of n using p processors, denoted $T(P)$.

Parallel Performance

Speed up

Speed up (S) measures how much faster a parallel program runs compared to its sequential version.

$$S_p = \frac{T(S)}{T(P)}$$

Efficiency

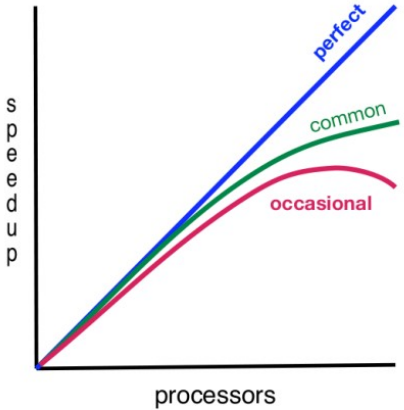
Measures how effectively processors are utilized

$$E_p = \frac{S_p}{p} = \frac{T(S)}{p \cdot T(P)}$$

Or as a percentage:

$$E_p = \frac{S_p}{p} \times 100\%$$

In The Real World



The Fundamental Law - Amdahl's Law

Statement

The overall performance improvement gained by optimizing a single part of a system is limited by the fraction of time that the improved part is actually used.

Examples

Let f be the fraction of time spent on serial operations. This make the parallel operation $1 - f$. Then for N processors:

$$Speedup = \frac{1}{(f + \frac{(1-f)}{N})}$$

Key Insight

The serial portion of your code limits the maximum speedup you can achieve, no matter how many processors you add!

Limitations of Parallel Processing

- Fraction of serial code limits parallel speedup
- Some problems cannot be broken down effectively
- Parallel tasks have different amounts of work
- Communication and synchronization overhead

The Reality

Effective speedup depends on minimizing communication and synchronization while maximizing parallel work.

The End